QUESTION 1

Title 1

Case Study: Log File Analysis System (Binary Search + Sorting)

Aim:

To design and analyze a Log File Analysis System that efficiently processes large log files using Sorting (Merge Sort/Quick Sort) and Binary Search, and to evaluate its performance using Divide and Conquer techniques.

Problem Statement:

Large log files generated by servers and applications contain thousands of entries. To quickly locate a specific event, log entries must first be sorted and then searched efficiently.

The system should:

- Sort log entries based on timestamp.
- Search for a specific event.
- Handle large datasets efficiently.
- Minimize processing time.

Divide and Conquer Approach:

Stage 1: Sorting

Merge Sort divides the log file into smaller parts, sorts them, and merges them.

Divide → Split logs
Conquer → Sort sublists
Combine → Merge sorted logs

Time Complexity:

O(n log n)

Stage 2: Binary Search

After sorting:

Divide → Find middle element
Conquer → Search left/right half

Time Complexity:

O(log n)

Key Issues:

Processing Time

Large log files require efficient sorting and searching.

Data Size

Millions of records cannot be searched sequentially efficiently.

Memory Usage

Merge Sort requires extra memory.

Solution Design:

1. Load log entries.
2. Sort entries using Merge Sort.
3. Apply Binary Search.
4. Return matching event.

Overall Time Complexity:

Sorting : O(n log n)
Binary Search: O(log n)

Total = O(n log n)

Sorting dominates the total complexity.

Justification:

The proposed solution is efficient because:

- Merge Sort guarantees O(n log n).
- Binary Search guarantees O(log n).
- Suitable for very large log datasets.
- Divide and Conquer improves scalability.

Code:

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
logs = [105, 101, 109, 103, 108, 102]
logs.sort()
target = 108
result = binary_search(logs, target)
if result != -1:
    print("Event Found at Index:", result)
else:
    print("Event Not Found")
```

Sample Output: Event Found at Index: 4

Result:

The Log File Analysis System successfully combines Merge Sort and Binary Search. Sorting organizes log data efficiently, while Binary Search enables rapid event retrieval. The overall complexity remains O(n log n), making the solution suitable for large-scale log analysis.